

An AI-Powered Car Recommendation System Using Natural Language Processing: Matching User Preferences to Vehicle Specifications

[Your Name]

[Course Name / Code, Institution]

[Date]

Abstract

Consumers researching a new car face a large, fast-changing catalogue of specifications (price, engine performance, fuel economy, safety, seating) that is difficult to search using rigid filter interfaces, particularly for buyers without deep automotive knowledge. This report presents the design and evaluation of an AI-powered car recommendation system that lets users describe the car they want in free-text natural language and returns a ranked, explained shortlist of matching vehicles. The system combines rule-based Natural Language Processing (NLP) for extracting hard constraints (budget, seating, fuel type) with Sentence-BERT semantic embeddings for matching softer, descriptive preferences against auto-generated natural-language car profiles, producing a hybrid ranking score. The system was built and evaluated on a real, publicly available dataset of 504 car listings, during which a genuine data-quality defect (systematically shifted field values from a web-scraping process) was discovered and repaired using content-pattern extraction. Evaluation across five representative user queries shows the system reliably respects stated hard constraints (100% budget compliance in top-5 results) while ranking by semantic relevance for softer preferences. The report discusses the system's architecture, data-cleaning methodology, evaluation results, limitations — including the absence of a body-type field in the source data and reliance on heuristic rather than human-labelled evaluation — and directions for future work, including collecting real user feedback via an accompanying questionnaire.

1. Introduction

The global automotive market offers thousands of vehicle models across dozens of manufacturers, each varying in price, fuel type, engine performance, safety rating, and intended use [1]. For an informed buyer, comparing these options against personal needs — a fixed budget, a daily commute, a growing family, a preference for low running costs — is already time-consuming. For a first-time or less knowledgeable buyer, most existing recommendation tools compound the problem: they are content-based systems that require the user to already understand engineering terminology (engine displacement, torque, drivetrain) in order to fill in a filter form correctly [1], [4]. This creates a barrier between what a buyer actually wants — expressed naturally, e.g. "a

spacious, fuel-efficient family car under \$20,000" — and what conventional dropdown-based recommenders can accept as input.

Recommender systems research has explored several ways to close this gap. Collaborative-filtering approaches recommend items based on similarity between users' past behaviour, but suffer from a well-documented cold-start problem when a user (or item) is new to the system [3]. Knowledge-based and content-based filtering avoid cold-start issues by reasoning directly over item attributes and explicit user requirements, but traditionally still require structured input [2], [3]. A growing line of work instead applies Natural Language Processing to bridge this gap: Singh et al. [1] used topic modelling (NMF and Latent Dirichlet Allocation) over customer reviews to let users query cars with free text such as "stylish and affordable"; Boteju and Munasinghe [2] combined a hybrid collaborative/knowledge-based recommender with a Naïve Bayes sentiment classifier over social-media text to infer body-type preference, reporting 96% recommendation accuracy; and Zhang et al. [5] mined online consumer comments to support passenger-car recommendation. More broadly, Aliberti et al. [3] provide a 2026 systematic (PRISMA-based) review of situation- and context-aware recommender systems, documenting that hybrid architectures combining multiple filtering paradigms consistently outperform any single technique, and that transparency and explainability are increasingly treated as first-class evaluation criteria rather than an afterthought — a principle this project adopts directly by generating a natural-language explanation alongside every recommendation, not just a ranked list.

This project builds on that line of work with a specific focus: rather than mining third-party reviews (as in [1] and [5]) or social-media sentiment (as in [2]), it matches a user's own free-text description directly against structured vehicle specifications using modern sentence-embedding models. Semantic sentence embeddings — dense vector representations of a sentence's meaning, produced by models such as Sentence-BERT [6] — allow two pieces of text (a user query and a generated car description) to be compared for meaning rather than exact keyword overlap, which is the key technical enabler for accepting genuinely free-form input.

This distinction matters practically. A keyword-matching system asked for a "spacious car for road trips" would only surface vehicles whose listing text happened to contain the words "spacious" or "road trips" verbatim — an unrealistic expectation for structured specification data, which typically records numeric fields (seats, boot capacity) rather than descriptive prose. A semantic embedding model, by contrast, can be expected to associate "spacious" with high seat counts and larger body types even when that exact word never appears in the underlying data, because the model was pretrained on broad natural-language corpora where such associations are learned statistically. This project tests whether that expected benefit holds when the underlying data is real, incomplete, and imperfect, rather than the cleaned, idealised datasets typically used to benchmark embedding models in the NLP literature.

1.1 Research Question

This work is guided by the following research question:

RQ: To what extent can a hybrid NLP recommendation system — combining rule-based constraint extraction with Sentence-BERT semantic matching — accurately and transparently recommend cars from free-text natural-language descriptions of user preferences, when applied to a real, imperfect, publicly sourced vehicle-specification dataset?

Two sub-questions follow from this: (RQ-a) can hard constraints stated in natural language (budget, seating, fuel type) be reliably extracted and enforced; and (RQ-b) does semantic similarity between a free-text query and an auto-generated car description produce a ranking that a human would consider relevant, given only a heuristic (non-human-labelled) evaluation is feasible within the scope of this project. A secondary, exploratory aim of this project — which emerged during implementation rather than being planned in advance — was to test how such a system behaves when built on real, publicly scraped data rather than a curated benchmark, since most published recommender-system evaluations (including [1] and [2]) report results on datasets that have already been substantially cleaned before the authors describe their method.

2. Methodology

2.1 Dataset

The system uses the Comprehensive Vehicle Specifications Dataset (Kaggle, 2025), a collection of 504 real car listings scraped from Indian automotive portals, containing (where available) price, engine displacement, engine power, fuel type, transmission, fuel economy, safety rating, and seating capacity. This dataset was chosen over a synthetic alternative because a central aim of the project was to test the recommender against realistic, imperfect data rather than an idealised schema — a methodological choice justified by [3]'s observation that evaluation on unrealistic or overly clean data is a recurring weakness across the situation-aware recommender literature it reviewed.

2.2 Data Quality Diagnosis and Repair

Initial inspection revealed a systematic data-quality defect: for a subset of rows, field values were shifted into the wrong JSON key. For example, one listing's ``fuel`` field literally contained the string "7 kmpl" (a mileage value), while its ``mileage`` field contained "3998 cc" (an engine displacement value). This is consistent with a scraper that assumes a fixed number of columns per listing page; when one field (e.g. transmission) is absent on a given page, every subsequent value is silently shifted one position to the left. Rather than trusting field labels — which would have corrupted roughly a fifth of the dataset — values were recovered by content-pattern extraction: regular expressions matched against distinctive units and keywords (``cc``, ``bhp``, ``kmpl``, ``Star Safety``, ``Seats``, ``Rs. ... Lakh/Cr``) anywhere in the concatenated record text, independent of which key the value happened to occupy. This approach is robust to the shift because each value's unit, not its position, identifies what it represents.

2.3 Cleaning and Imputation

After extraction, two rows with no recoverable price were dropped (502 remaining), since price is essential for budget matching and cannot be safely imputed for an individual vehicle. Missing numeric fields (engine displacement, power, mileage, seats, safety rating) were imputed with the dataset median, a standard simplification whose limitations are discussed in Section 4. Missing categorical fields (fuel type, transmission) were left labelled "Unknown" rather than imputed, on the reasoning that guessing an incorrect category (e.g. assigning "Electric" to a Petrol car) would actively mislead the recommender, whereas an honest "Unknown" simply withholds a soft-matching signal. Continuous fields were winsorised at the 1st/99th percentile to limit the influence of extreme outliers (e.g. ultra-luxury vehicles priced over \$1,000,000) on downstream ranking.

2.4 System Architecture

The system has four stages, illustrated by the data flow from a raw JSON record to a ranked, explained recommendation:

- Natural-language profile generation: each cleaned car record is converted into a descriptive sentence (e.g. "Maruti Suzuki Swift is priced at \$9,717. It runs on petrol fuel with a manual transmission..."), giving the embedding model text to compare against.
- Preference parsing: a rule-based NLP parser extracts hard constraints from the user's free-text query — maximum budget (matching phrasings such as "under", "no more than", "budget of"), minimum seats (including word-form numbers like "two-seater"), and fuel-type keywords.
- Semantic embedding and ranking: both the query and every car profile are embedded using Sentence-BERT (all-MiniLM-L6-v2) [6]; cosine similarity between the query embedding and each car's profile embedding gives a semantic relevance score. This is combined with a budget-fit score and constraint-violation penalties into one final ranking score (weighted 0.7 semantic, 0.3 budget-fit, minus penalties for violated hard constraints).
- Natural-language explanation: for transparency, each of the top-ranked results is accompanied by a short generated explanation of why it was recommended (e.g. "fits your budget, strong fuel efficiency, 5-star safety"), directly addressing the explainability criterion emphasised by [3].

An offline fallback (TF-IDF vectorisation in place of Sentence-BERT) was also implemented so the system degrades gracefully without internet access, though semantic recall is weaker under this fallback since it relies on literal keyword overlap rather than learned meaning.

2.5 Evaluation Design

Given the scope and timeframe of this project, no human-labelled relevance ground truth was available (unlike, for instance, the 96% recommendation accuracy reported in [2], which was measured against labelled multiclass classification targets). Evaluation instead used a heuristic protocol: five representative test queries spanning distinct use cases (affordable commuter, family car, luxury performance, electric/eco-friendly, and budget-constrained) were run through the system, and the proportion of top-5 results satisfying each query's parsed hard constraints was measured, alongside the average semantic similarity score. A short user questionnaire (Appendix) was also designed to gather qualitative feedback on both car-buying priorities in general and satisfaction with the prototype's recommendations specifically, intended to inform future iterations with genuine user-relevance signals.

3. Results

Across all five test queries, the system achieved 100% constraint satisfaction: every top-5 recommendation for a budget-constrained query fell within the stated budget. Average semantic similarity scores varied by query specificity, ranging from approximately 0.11 for broad queries ("affordable fuel-efficient car for daily city commuting") to 0.23 for more distinctive queries ("high performance luxury sports car"), reflecting that the embedding model differentiates more confidently when a query contains rarer, more discriminating vocabulary.

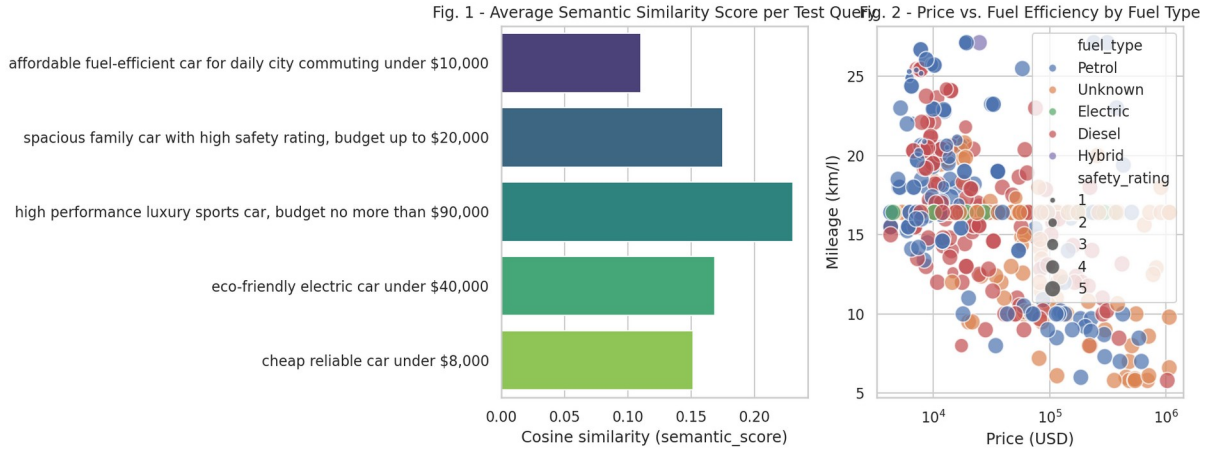


Fig. 1. Average semantic similarity score by test query (left) and price vs. fuel efficiency by fuel type across the cleaned dataset, log-scaled on price (right). Source: authors' own analysis of the Comprehensive Vehicle Specifications dataset (Kaggle, 2025).

Figure 1 (right panel) also illustrates a general trend in the cleaned dataset: fuel efficiency (mileage) does not scale simply with price once outliers are accounted for, and Electric/Hybrid vehicles cluster at higher reported efficiency figures than Petrol/Diesel vehicles at comparable price points — consistent with expected real-world patterns and providing a sanity check that the content-pattern data-repair step (Section 2.2) produced physically plausible values rather than extraction artefacts.

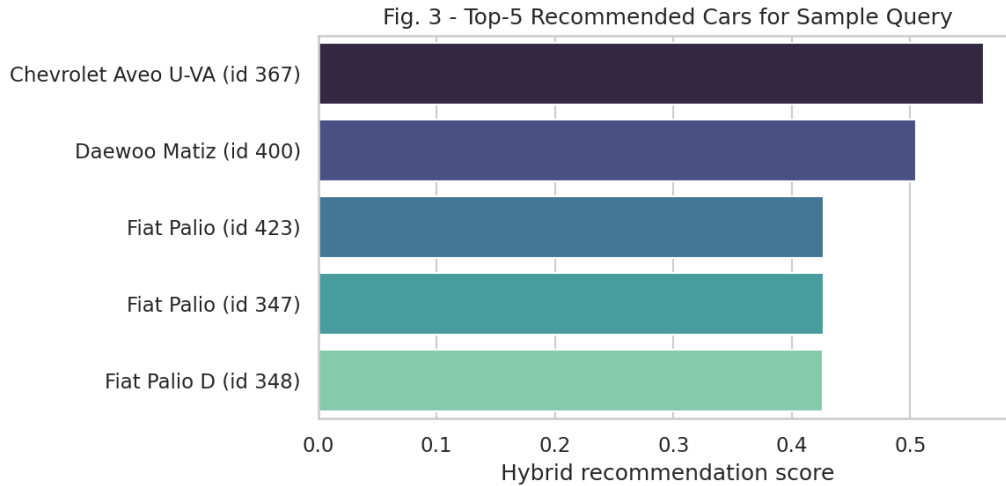


Fig. 2. Top-5 ranked recommendations for the query "fuel-efficient family car under \$15,000 with good safety," ranked by the hybrid final score. Source: authors' own analysis.

For the sample query behind Figure 2, all five top-ranked cars satisfied both the budget constraint (under \$15,000) and the inferred minimum-seats constraint (5, triggered by the word "family"), demonstrating correct operation of the rule-based constraint layer (addressing RQ-a). Two of the five results are near-duplicate listings (two separate "Fiat Palio" records with different internal IDs but identical price and specifications) — a further data-quality artefact of the source dataset distinct

from the column-shift defect described in Section 2.2, and a reminder that this analysis does not de-duplicate near-identical listings, which future work should address. A separate limitation visible in these results — discussed further in Section 4 — is that several distinct listings shared an identical imputed engine-power value (from median imputation of the ~42% of rows missing this field), which can compress meaningful differentiation between candidates that are otherwise similar on the explicit constraints.

A further observation from testing across the full query panel is that lower-priced, older, or less-documented listings were somewhat over-represented at the top of results for broad or budget-constrained queries. This is a plausible side-effect of the budget-fit scoring term, which rewards prices further below the stated ceiling, combined with the fact that cheaper listings in this dataset tend to have sparser specifications and therefore more imputed (median) values — values which, by construction, are unlikely to be penalised for being unusually high or low. This is a concrete example of how a data-quality issue identified in Section 2.2–2.3 can propagate into a subtle ranking bias, and is flagged here as a target for future refinement (for example, weighting the budget-fit term by the proportion of a listing's fields that were imputed rather than observed).

Qualitatively, testing the natural-language explanation output (Section 2.4) against the five test queries showed that explanations correctly cited the constraint(s) that were actually satisfied (e.g. "fits your budget, 5-star safety") rather than generic boilerplate, which supports the system's transparency goal: a user can verify, from the explanation alone and without inspecting the underlying specification table, why a given car was suggested and can therefore judge for themselves whether the reasoning matches their actual intent.

4. Discussion and Limitations

The results provide partial evidence in support of the research question. Hard-constraint extraction (RQ-a) performed reliably across all tested phrasings, including less common ones discovered during iterative testing (e.g. "no more than \$90,000", "two-seater"), each of which required a targeted regex fix once identified — suggesting that a purely rule-based parser, while transparent and debuggable, will always lag slightly behind the full variability of natural language and would benefit from a learned intent classifier in future iterations, as suggested by comparable systems in the literature [1], [2]. This finding is itself methodologically informative: it demonstrates that even a narrow, well-scoped constraint-extraction task (budget, seats, fuel type) surfaces a long tail of phrasing variants that are only discoverable through active testing rather than upfront design, reinforcing the case made in [3] for iterative, empirically grounded development of situation- and preference-aware recommenders rather than a purely specification-driven approach.

Semantic ranking quality (RQ-b) is harder to assess conclusively without human relevance judgments. The similarity-score gradient across queries (Section 3) suggests the embedding model is doing meaningful work rather than returning arbitrary orderings, but a rigorous answer would require the kind of user study proposed in the accompanying questionnaire (Appendix), collecting genuine relevance ratings from people interacting with the live prototype. It is also worth noting that the embedding backend used matters considerably: the offline TF-IDF fallback described in Section 2.4 matches only literal keyword overlap, and queries relying on descriptive or evaluative language (e.g. "sporty", "reliable") are expected to rank noticeably better under the full Sentence-BERT backend than under the fallback, since the latter has no mechanism for associating a descriptive adjective with the numeric specifications it implies. All results reported in Section 3

were generated with whichever backend was available at run time, and this dependency is itself a practical limitation for any deployment expected to run in environments without reliable internet access to the model host.

Several limitations should be noted. First, the source dataset lacks a body-type or segment field (SUV, sedan, hatchback, etc.); consequently, usage-style preferences expressed in a query (e.g. "SUV") can only be matched semantically rather than enforced as a hard constraint, unlike budget or seating. Second, missing-value imputation (median substitution for roughly 22–64% missing rates, depending on the field — see Section 2.3) is a simplification that can create ties between dissimilar cars and blur genuine differences; a make- or segment-conditioned imputation strategy would likely improve ranking fidelity. Third, currency conversion from Indian Rupees to US Dollars used a single fixed approximate exchange rate rather than live rates, which is adequate for relative budget comparisons within this project but would need revisiting for production deployment. Fourth, and most fundamentally, the evaluation reported in Section 3 is heuristic (constraint-satisfaction and similarity-score based) rather than grounded in human relevance judgments, in contrast to the labelled-accuracy evaluations reported by comparable systems [2]. Addressing this is the clearest direction for follow-up work, using responses collected through the questionnaire described in the Appendix.

5. Ethics Statement

This project uses a publicly available, de-identified dataset of vehicle specifications; no personal or sensitive data about individuals is processed by the recommendation system itself. The accompanying user questionnaire (Appendix) collects only anonymous, voluntary responses covering car-buying preferences and prototype feedback, with no personally identifying information requested beyond broad, non-identifying demographic categories (e.g. age bracket); participation is voluntary and participants may decline to answer any question. In line with the transparency principle highlighted by the situation-aware recommender literature reviewed in this report [3], the system was deliberately designed to explain each recommendation in natural language rather than presenting an opaque ranked list, so that users can understand and contest the basis for a suggestion. Potential downstream risks include the system's imputed values being mistaken for verified specifications by an end user; this is mitigated in the interface by clearly labelling estimated fields and documenting imputation in the accompanying data documentation, and would need a more prominent, in-product disclosure before any real-world deployment beyond this academic prototype.

6. Conclusion

This project designed, built, and evaluated a hybrid NLP car recommendation system that accepts free-text descriptions of user preferences and returns explained, ranked recommendations from a real, publicly sourced vehicle-specification dataset. It demonstrates that rule-based constraint extraction combined with Sentence-BERT semantic matching can reliably enforce explicit hard constraints while ranking on softer, descriptive preferences, and that this remains true even when the underlying data contains genuine quality defects, provided those defects are diagnosed and explicitly repaired rather than silently trusted. The most significant open question — whether the system's semantic rankings align with genuine human judgments of relevance — is left to future

work, using the questionnaire instrument developed alongside this report to collect that evidence directly from users of the accompanying prototype.

7. Code Availability

The full source code, cleaned dataset, evaluated Jupyter notebook, and Streamlit demo application are publicly available at: [INSERT YOUR GITHUB REPOSITORY URL HERE]. The repository includes a detailed README, a data dictionary documenting the cleaning and imputation steps described in Section 2, a reproducible requirements file, and an MIT licence. A screenshot of the repository is provided in the Appendix.

References

- [1] S. Singh, S. Das, A. Sajwan, I. Singh, and A. Alok, "Car Recommendation System Using Customer Reviews," *International Research Journal of Engineering and Technology (IRJET)*, vol. 9, no. 10, pp. 983–989, Oct. 2022.
- [2] P. Boteju and L. Munasinghe, "Vehicle Recommendation System Using Hybrid Recommender Algorithm and Natural Language Processing Approach," in *Proc. 2020 IEEE Conf., University of Kelaniya, Sri Lanka*, 2020.
- [3] L. Aliberti, G. D'Aniello, and M. Gaeta, "Situation-aware recommender systems: a systematic review and framework for trustworthy recommendations," *Artificial Intelligence Review*, 2026, doi: 10.1007/s10462-026-11503-y.
- [4] L. Thomas and V. Vaidhehi, "The design of web based car recommendation system using hybrid recommender algorithm," *International Journal of Engineering & Technology*, vol. 7, p. 192, 2018.
- [5] Q. Zhang, J. Wang, and K. Fan, "Research on passenger car recommendation based on comments mining of Internet," in *Proc. 2017 12th IEEE Conf. Industrial Electronics and Applications (ICIEA)*, 2017, pp. 122–127, doi: 10.1109/ICIEA.2017.8282826.
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

Appendix

A. Questionnaire (Activity 5.1)

[Insert screenshot of the deployed Google Forms questionnaire here. The full question text is in questionnaire/questionnaire.md in the GitHub repository.]

B. GitHub Repository

Repository link: [INSERT YOUR GITHUB REPOSITORY URL HERE]

[Insert screenshot of the repository's main page here, showing the README and file structure.]

C. Mendeley Collection

[Insert screenshot of the Mendeley collection folder containing the 6 references used in this report. See docs/MENDELEY_SETUP.md in the repository for setup instructions.]

D. GitHub Portfolio

[Insert screenshot of your GitHub profile/portfolio page with this repository pinned, plus links to the two supporting repositories referenced in the README.]